29.3 Calling your web APIs from other domains using CORS

In this section you'll learn about cross-origin resource sharing (CORS), a protocol to allow JavaScript to make requests from one domain to another. CORS is a frequent area of confusion for many developers, so this section describes why it's necessary and how CORS headers work. You'll then learn how to add CORS to both your whole application and specific web API actions, and how to configure multiple CORS policies for your application.

As you've already seen, CSRF attacks can be powerful, but they would be even more dangerous if it weren't for browsers implementing the *same-origin policy*. This policy blocks apps from using JavaScript to call a web API at a different location unless the web API explicitly allows it.

DEFINITION *Origins* are deemed to be the same if they match the scheme (HTTP or HTTPS), domain (example.com), and port (80 by default for HTTP and 443 for HTTPS). If an app attempts to access a resource using JavaScript, and the origins aren't identical, the browser blocks the request.

The same-origin policy is strict. The origins of the two URLs must be identical for the request to be allowed. For example, the following origins are the same:

- http://example.com/home
- http://example.com/site.css

The paths are different for these two URLs (/home and /site.css), but the scheme, domain, and port (80) are identical. So if you were on the home page of your app, you could request the /site.css file using JavaScript without any problems.